

Chumeng' s Code Library

chumeng

October 26, 2025

Contents

一切的开始	2
VIM 赛时配置	2
VIM 快速编译	2
对拍	2
随机数	2
快读快写	2
数据结构	3
ST 表	3
数学	4
类欧几里得	4
图论	4
LCA	4
计算几何	4
二维几何：点与向量	4
字符串	6
后缀自动机	6
杂项	6
STL	6

一切的开始

VIM 赛时配置

```
sy on
set rnu ts=4 sw=4 cin
colo slate
```

VIM 快速编译

```
set makeprg=g++\ %\ -o\ %<\ -std=c++23\ -fsanitize=undefined\ -g
:make 编译
```

对拍

```
1  #!/usr/bin/env bash
2  g++ -o r main.cpp -O2 -std=c++23
3  g++ -o std std.cpp -O2 -std=c++23
4  while true; do
5      python gen.py > in
6      ./r < in > out
7      ./std < in > stdout
8      if test $? -ne 0; then
9          exit 0
10     fi
11     if diff stdout out; then
12         printf "AC\n"
13     else
14         printf "WA\n"
15         exit 0
16     fi
17 done
```

随机数

```
1  #include <random>
2
3  std::mt19937 rng(std::chrono::steady_clock::now().time_since_epoch().count());
4  std::uniform_int_distribution<int> dist(1, 100);
5
6  int main() {
7      int x = dist(rng);
8      printf("%d\n", x);
9      return 0;
10 }
```

快读快写

```
1  namespace IO {
2      char buf[1 << 23], *p1 = buf, *p2 = buf;
3      #define getchar() (p1 == p2 && (p2 = (p1 = buf) + fread(buf, 1, 1 << 21, stdin), p1 == p2) ? EOF : *p1++)
4      template <typename T>
5      inline T read() {
6          char c = getchar();
7          T res = 0, sig = 1;
8          while (!std::isdigit(c)) {
9              if (c == '-') {
10                 sig = -1;
11             }
12             c = getchar();
13         }
14         while (std::isdigit(c)) {
15             res = res * 10 + c - '0';
16             c = getchar();
17         }
18         return res * sig;
19     }
```

```

20 template <typename T>
21 inline void write(T x) {
22     if (!x) {
23         putchar('0');
24         return;
25     }
26     static int stk[100], tail = 0;
27     if (x < 0) {
28         x = -x;
29         putchar('-');
30     }
31     while (x) {
32         stk[tail++] = x % 10;
33         x /= 10;
34     }
35     for (int i = --tail; i >= 0; i--) {
36         putchar(stk[i] + '0');
37     }
38 }
39 template <typename T>
40 inline void read(T& x) {
41     x = read<T>();
42 }
43 template <typename T, typename... Args>
44 inline void read(T& x, Args&... args) {
45     read(x);
46     read(args...);
47 }
48 struct reader {
49     template <typename T>
50     reader& operator>>(T& x) {
51         if constexpr (std::is_integral_v<T> && !std::is_same_v<std::decay_t<T>, char>) {
52             read(x);
53         } else {
54             std::cout << x;
55         }
56         return *this;
57     }
58 } cin;
59 struct writer {
60     template <typename T>
61     writer& operator<<(T x) {
62         if constexpr (std::is_integral_v<T> && !std::is_same_v<std::decay_t<T>, char>) {
63             write(x);
64         } else {
65             std::cout << x;
66         }
67         return *this;
68     }
69 } cout;
70 }; // namespace IO

```

数据结构

ST 表

- 二维

```

1 int f[maxn][maxn][10][10];
2 inline int highbit(int x) { return 31 - __builtin_clz(x); }
3 inline int calc(int x, int y, int xx, int yy, int p, int q) {
4     return max(
5         max(f[x][y][p][q], f[xx - (1 << p) + 1][yy - (1 << q) + 1][p][q]),
6         max(f[xx - (1 << p) + 1][y][p][q], f[x][yy - (1 << q) + 1][p][q])
7     );
8 }
9 void init() {
10     FOR (x, 0, highbit(n) + 1)
11     FOR (y, 0, highbit(m) + 1)
12     FOR (i, 0, n - (1 << x) + 1)

```

```

13     FOR (j, 0, m - (1 << y) + 1) {
14         if (!x && !y) { f[i][j][x][y] = a[i][j]; continue; }
15         f[i][j][x][y] = calc(
16             i, j,
17             i + (1 << x) - 1, j + (1 << y) - 1,
18             max(x - 1, 0), max(y - 1, 0)
19         );
20     }
21 }
22 inline int get_max(int x, int y, int xx, int yy) {
23     return calc(x, y, xx, yy, highbit(xx - x + 1), highbit(yy - y + 1));
24 }

```

数学

类欧几里得

- $m = \lfloor \frac{an+b}{c} \rfloor$.
- $f(a, b, c, n) = \sum_{i=0}^n \lfloor \frac{ai+b}{c} \rfloor$: 当 $a \geq c$ or $b \geq c$ 时, $f(a, b, c, n) = (\frac{a}{c})n(n+1)/2 + (\frac{b}{c})(n+1) + f(a \bmod c, b \bmod c, c, n)$; 否则 $f(a, b, c, n) = nm - f(c, c-b-1, a, m-1)$ 。
- $g(a, b, c, n) = \sum_{i=0}^n i \lfloor \frac{ai+b}{c} \rfloor$: 当 $a \geq c$ or $b \geq c$ 时, $g(a, b, c, n) = (\frac{a}{c})n(n+1)(2n+1)/6 + (\frac{b}{c})n(n+1)/2 + g(a \bmod c, b \bmod c, c, n)$; 否则 $g(a, b, c, n) = \frac{1}{2}(n(n+1)m - f(c, c-b-1, a, m-1) - h(c, c-b-1, a, m-1))$ 。
- $h(a, b, c, n) = \sum_{i=0}^n \lfloor \frac{ai+b}{c} \rfloor^2$: 当 $a \geq c$ or $b \geq c$ 时, $h(a, b, c, n) = (\frac{a}{c})^2n(n+1)(2n+1)/6 + (\frac{b}{c})^2(n+1) + (\frac{a}{c})(\frac{b}{c})n(n+1) + h(a \bmod c, b \bmod c, c, n) + 2(\frac{a}{c})g(a \bmod c, b \bmod c, c, n) + 2(\frac{b}{c})f(a \bmod c, b \bmod c, c, n)$; 否则 $h(a, b, c, n) = nm(m+1) - 2g(c, c-b-1, a, m-1) - 2f(c, c-b-1, a, m-1) - f(a, b, c, n)$ 。

图论

LCA

- 倍增

```

1 void dfs(int u, int fa) {
2     pa[u][0] = fa; dep[u] = dep[fa] + 1;
3     FOR (i, 1, SP) pa[u][i] = pa[pa[u][i-1]][i-1];
4     for (int& v: G[u]) {
5         if (v == fa) continue;
6         dfs(v, u);
7     }
8 }
9
10 int lca(int u, int v) {
11     if (dep[u] < dep[v]) swap(u, v);
12     int t = dep[u] - dep[v];
13     FOR (i, 0, SP) if (t & (1 << i)) u = pa[u][i];
14     FORD (i, SP-1, -1) {
15         int uu = pa[u][i], vv = pa[v][i];
16         if (uu != vv) { u = uu; v = vv; }
17     }
18     return u == v ? u : pa[u][0];
19 }

```

计算几何

二维几何：点与向量

```

1 #define y1 yy1
2 #define nxt(i) ((i + 1) % s.size())
3 typedef double LD;
4 const LD PI = 3.14159265358979323846;
5 const LD eps = 1E-10;
6 int sgn(LD x) { return fabs(x) < eps ? 0 : (x > 0 ? 1 : -1); }
7 struct L;
8 struct P;

```

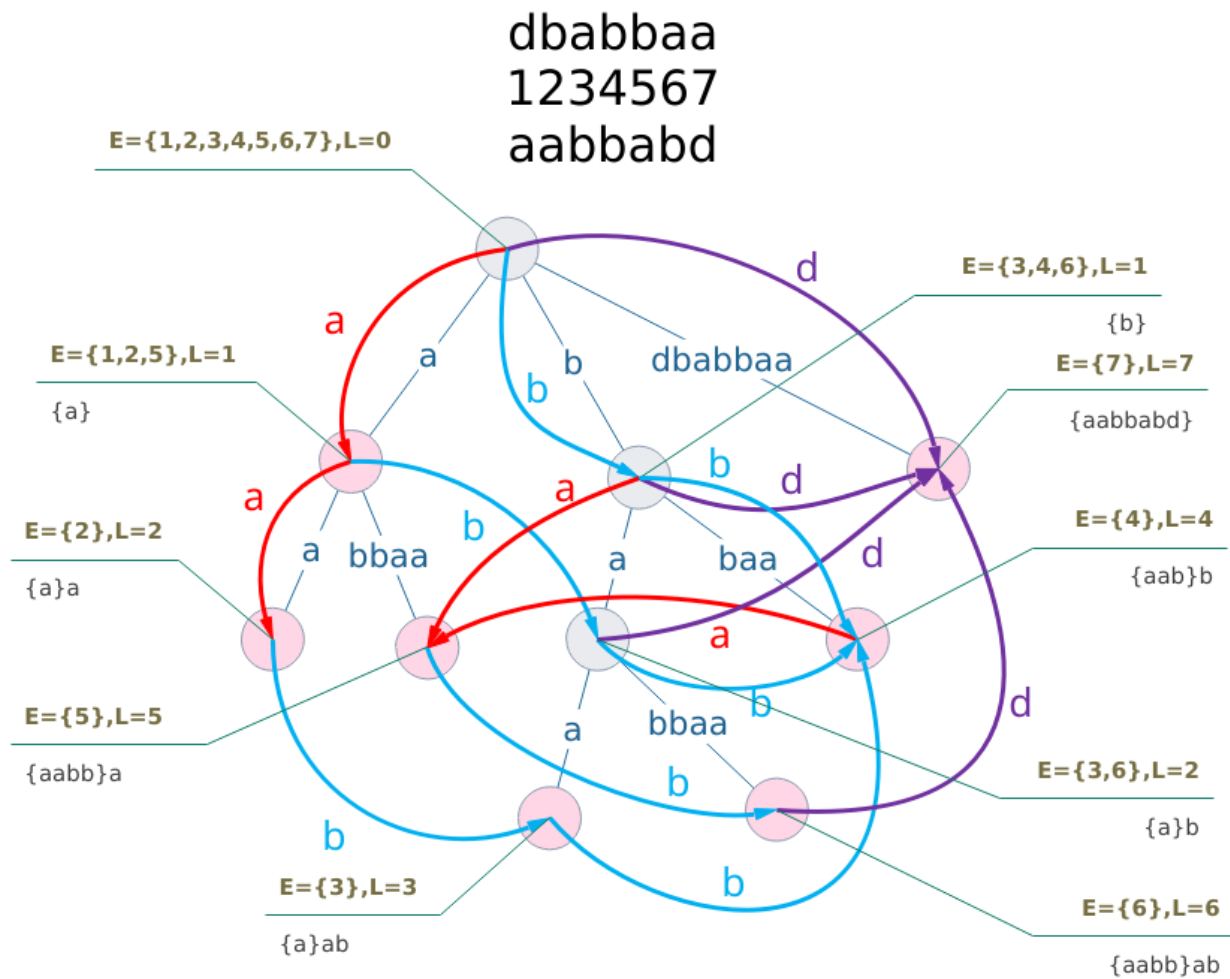
```

9  typedef P V;
10 struct P {
11     LD x, y;
12     explicit P(LD x = 0, LD y = 0): x(x), y(y) {}
13     explicit P(const L& l);
14 };
15 struct L {
16     P s, t;
17     L() {}
18     L(P s, P t): s(s), t(t) {}
19 };
20
21 P operator + (const P& a, const P& b) { return P(a.x + b.x, a.y + b.y); }
22 P operator - (const P& a, const P& b) { return P(a.x - b.x, a.y - b.y); }
23 P operator * (const P& a, LD k) { return P(a.x * k, a.y * k); }
24 P operator / (const P& a, LD k) { return P(a.x / k, a.y / k); }
25 inline bool operator < (const P& a, const P& b) {
26     return sgn(a.x - b.x) < 0 || (sgn(a.x - b.x) == 0 && sgn(a.y - b.y) < 0);
27 }
28 bool operator == (const P& a, const P& b) { return !sgn(a.x - b.x) && !sgn(a.y - b.y); }
29 P::P(const L& l) { *this = l.t - l.s; }
30 ostream &operator << (ostream &os, const P &p) {
31     return (os << "(" << p.x << ", " << p.y << ")");
32 }
33 istream &operator >> (istream &is, P &p) {
34     return (is >> p.x >> p.y);
35 }
36
37 LD dist(const P& p) { return sqrt(p.x * p.x + p.y * p.y); }
38 LD dot(const V& a, const V& b) { return a.x * b.x + a.y * b.y; }
39 LD det(const V& a, const V& b) { return a.x * b.y - a.y * b.x; }
40 LD cross(const P& s, const P& t, const P& o = P()) { return det(s - o, t - o); }
41 // -----

```

字符串

后缀自动机



杂项

STL

- copy

```
1 template <class InputIterator, class OutputIterator>  
2     OutputIterator copy (InputIterator first, InputIterator last, OutputIterator result);
```